

VEGA

Runbook Following Agent

Verify · Execute · Guide · Audit

A complete record of prompts used and the AI outputs
that shaped each stage of the autonomous agent

Rakasri L · Ramya Ruba S · Joha Shekin Saroj R · SivaSuriya

Department of CSE

VSB Engineering College, Karur

Prompt Engineering Documentation

Overview

This documentation presents the complete prompt-engineering trail used to design and build VEGA, an autonomous Runbook Following Agent. VEGA stands for Verify, Execute, Guide, and Audit — the four operating principles that govern every action it takes.

Objective

To record, in order, each prompt issued to the AI assistant, summarise the AI output that resulted, and trace how those outputs were composed into the final agent. The document is intended as a transparent reference for evaluation, reproducibility, and future extension.

How to read this document

- Each stage lists the Prompt Issued, an AI Output Summary, and the Objective met by that prompt.
- Prompts are numbered sequentially in the order they were used during construction.
- Outputs are summarised in plain language — full code listings live in the project repository.

Prompt 1 — Problem Framing and Initial Ideation

Objective

Establish the scope, mission, and high-level architecture of the Runbook Following Agent before any code is written.

Prompt Issued

"

Help me design an autonomous DevOps agent called VEGA that reads YAML runbooks and executes their steps end-to-end. It must support shell, SQL, REST, Python, and human-approval steps, classify every action as safe / needs_approval / blocked, notify operators on Discord, and persist execution history. Propose the architecture, technology stack, and a staged build plan.

AI Output Summary

The AI returned a four-principle framing (Verify, Execute, Guide, Audit), a stack of React 19 + Vite frontend, Flask + SQLAlchemy backend, Ollama with Llama3 as the reasoning model, SQLite for persistence, and Discord webhooks for notifications. It outlined a six-stage build plan: scaffold, runbook parser, step executors, safety classifier, notifier, and frontend UI. It also recommended a MOCK_OLLAMA flag for deterministic tests.

Prompt 2 — Master System Prompt for Code Generation

Objective

Lock in the conventions the assistant must follow for every subsequent code-generation prompt so the project stays internally consistent.

Prompt Issued

"

Write the master system prompt for the VEGA code generator. It must enforce a strict directory layout (backend/, frontend/, runbooks/, tests/), require MOCK_OLLAMA=true in tests, mandate an AllowList class containing SAFE_COMMANDS and CLOUD_COMMANDS sets, and define an Approval table for human-in-the-loop steps. The prompt must forbid destructive defaults and require explicit allow-listing.

AI Output Summary

The AI produced a system prompt that fixes directory conventions, names the AllowList class with its two allow-listed sets, defines the Approval SQLAlchemy model (id, step_id, status, requested_by, decided_by, reason, timestamps), and lists hard rules: no rm -rf, no DROP without approval, every executor returns a structured result, and tests must run offline with MOCK_OLLAMA=true.

Prompt 3 — Stage 1 — Project Scaffold

Objective

Bootstrap the repository with working frontend, backend, and database wiring so later stages can plug in features.

Prompt Issued

"

Generate the Stage 1 scaffold: a Flask backend with SQLAlchemy and a /health route, a Vite + React 19 frontend that pings the backend, a SQLite database initialised on first run, and a Makefile with dev, test, and seed targets.

AI Output Summary

The AI delivered a runnable monorepo: backend/app.py with the Flask factory pattern, backend/models.py with the initial Runbook and Execution tables, a Vite frontend that renders a status card from /health, and a Makefile that starts both services concurrently. The seed target inserts one demo runbook.

Prompt 4 — Stage 2 — Runbook Parser and Step Executors

Objective

Turn declarative YAML runbooks into typed step objects and execute each step type through a dedicated, sandboxed executor.

Prompt Issued

"

Implement the YAML runbook parser and five executors: ShellExecutor (subprocess with timeout and AllowList check), SQLExecutor (parameterised queries only), RESTExecutor (requests with retry and timeout), PythonExecutor (restricted globals), and ApprovalExecutor (creates an Approval row and blocks until decided). Every executor must return {status, stdout, stderr, duration_ms, artifacts}.

AI Output Summary

The AI generated parser.py producing typed Step dataclasses, an executors/ package with the five classes sharing a common BaseExecutor, and a dispatcher that maps step.type to the right executor. ShellExecutor rejects any command not in SAFE_COMMANDS unless an Approval row is in status="approved". All executors share a uniform result schema, making downstream logging trivial.

Prompt 5 — Stage 3 — Safety Classifier and Reasoner

Objective

Add an LLM-based safety layer in front of execution and a reasoning layer that decides what to do when a step fails.

Prompt Issued

"

Write the Ollama system prompt for the safety classifier. Given a step description, it must return strict JSON: {"verdict": "safe" | "needs_approval" | "blocked", "reason": string}. Then write the runbook reasoner prompt that, given the full step history, returns {"decision": "retry" | "branch" | "abort" | "request_approval", "rationale": string}.

AI Output Summary

The AI produced two tightly scoped system prompts. The classifier prompt fixes the JSON schema, forbids prose, and lists examples for each verdict. The reasoner prompt injects the ordered step history, the last

error, and the runbook goal, then constrains the model to one of four decisions with a one-paragraph rationale. Both prompts are wrapped by a Python helper that validates the JSON and falls back to "blocked" / "abort" on parse failure.

Prompt 6 — Stage 4 — Discord Notifier and Approval Loop

Objective

Keep humans informed and in control by routing high-risk events and approval requests to Discord.

Prompt Issued

"

Build a Discord notifier that posts a rich embed on three events: step needs_approval, step blocked, and execution finished. Include a deep link back to the frontend approval page, and listen for an /approve or /deny slash response that updates the Approval row.

AI Output Summary

The AI generated notifier.py using discord.py webhook embeds with colour-coded statuses (amber for approval, red for blocked, green for success). The slash command handler verifies the requester is in an allow-listed Discord role before flipping the Approval row, then signals the blocked ApprovalExecutor to resume.

Prompt 7 — Stage 5 — Frontend Runbook UI

Objective

Give operators a clear, real-time view of runbooks, executions, and pending approvals.

Prompt Issued

"

Create the React 19 frontend: a runbook list, a run-detail page showing each step with live status (pending, running, ok, failed, awaiting_approval), an approval panel with approve / deny buttons, and a log drawer that streams stdout / stderr.

AI Output Summary

The AI shipped pages for /runbooks, /runs/:id, and /approvals, using React Query for polling and a small EventSource hook for log streaming. Step rows colour-code by status and expand to reveal the executor result. The approval panel calls /api/approvals/:id with the operator decision and rationale.

Prompt 8 — Stage 6 — Traceability and Audit

Objective

Make every decision the agent took auditable after the fact.

Prompt Issued

"

Add an audit log that records, for every step, the classifier verdict, the reasoner decision (if invoked), the executor result, and any approval metadata. Expose it as `/api/runs/:id/audit` and render a downloadable JSON on the run-detail page.

AI Output Summary

The AI added an AuditEvent table, wrote into it from every executor and from both LLM helpers, and added a frontend download button. The audit JSON groups events by step and includes timestamps, the exact prompt sent to Ollama, and the raw model response — closing the loop on the Audit principle.

Traceability Matrix

The matrix below maps each prompt to the feature it produced and the VEGA principle it reinforces.

Prompt #	Stage / Feature	VEGA Principle
1	Problem framing	Verify
2	Master system prompt	Verify · Audit
3	Project scaffold	Execute
4	Parser + executors	Execute
5	Safety classifier + reasoner	Verify · Guide
6	Discord notifier + approvals	Guide
7	Frontend runbook UI	Guide
8	Audit log	Audit

Closing Notes

Every prompt above was issued to a single AI assistant working under the master system prompt defined in Prompt 2. Outputs were reviewed, lightly edited for project conventions, and committed alongside the tests they enabled. The result is an autonomous agent whose behaviour can be retraced — from a YAML runbook step, through the safety verdict, the reasoner decision, the executor result, and the operator approval — back to the original prompt that shaped each layer.